VEGA

Vision, Current Position, and Resource Proposal

v1.0 | Prepared for review | September 2026

VEGA is an assistant that does not merely answer questions — it operates the machine. It manages work, creates and organises files, produces documents, drives a web browser, fills forms, reports on the system, sends email, and accepts instructions by voice. A working version exists today and has been verified in daily use.

Purpose of this document. It sets out what the product is for and who it serves, states where development is now blocked, records the approaches already taken to avoid that block, explains why cloud compute does not resolve it, and proposes the specific resources required. Implementation detail is intentionally excluded.

1. Vision and purpose

Assistants today are overwhelmingly *advisory*. They explain how to do something and leave you to do it. The gap between knowing and doing is where a working day is actually spent.

Mission. To close that gap — an assistant that carries out the work rather than describing it, that runs on infrastructure its owner controls, and that can be trusted because it never claims to have done something it has not done.

Who it is for. Professionals and technical teams who spend a substantial part of the day on machine operations — moving and organising files, producing documents, filling forms, checking systems, handling email, automating repetitive workflow. The commercial case is any organisation that wants that work automated but cannot allow its data to leave its own network.

What separates it. Two properties, both of which are difficult for a hosted competitor to match:

Property	Why it matters commercially
It runs on infrastructure the customer owns	No document, credential or instruction reaches a third party. For legal, medical, financial and defence-adjacent customers this is a procurement requirement rather than a preference, and it is something the large hosted providers structurally cannot offer.
It does not overstate what it did	An assistant that acts on your behalf and reports success falsely is more dangerous than one that fails visibly. The system holds a standing rule that an action must be confirmed before it is reported as done, and where confirmation is genuinely impossible it says so rather than implying success. This is enforced automatically.

2. Where the project stands

A complete working version exists. It is not a prototype or a demonstration script: it is in daily use, its behaviour is covered by an automated test suite and four standing audits, and it has been verified to install and run correctly on a machine that had never run it before.

Working today	Tasks and projects, team records, files and folders, Word and Excel document generation, browser automation including form filling, system and resource reporting, email, and voice control.
Quality position	514 automated tests, four standing audits, and a frozen benchmark that allows any change to be judged by measurement rather than opinion.

Honest gaps	Instructions combining several dependent steps succeed less than half the time. Understanding does not generalise to unfamiliar phrasing. Generated content is limited in quality.

Every one of those gaps has the same root cause, which is the subject of the next section.

3. Where development is blocked

The system runs on one consumer laptop. The graphics memory available for running language models is 4 GB. That single number is the origin of nearly every remaining weakness in the product.

Four gigabytes constrains the project to very small language models. Each item below is a measured consequence, not an expectation:

Observed problem	Consequence for the product
An instruction requiring the model to interpret it takes on the order of twelve seconds	The system feels slow at exactly the moments it is doing its most valuable work. This is the largest single barrier to it being pleasant to use, and the first thing a customer would notice.
The vision model does not fit in available graphics memory	Screen understanding falls back to a substantially weaker method.
Only one model can be held in memory at a time	Specialised roles cannot run concurrently. Work that could be parallel is forced into sequence, compounding the delay above.
The model cannot reliably produce working code or long structured output	An entire category of valuable work is out of reach.
Instructions with several dependent steps fail more often than they succeed	The system handles single instructions well and compound ones poorly, which is the difference between a tool and an assistant.

4. What was tried before requesting hardware

This request is made last, not first. Considerable engineering effort went into avoiding it, and that effort is what establishes that the remaining limit is hardware rather than technique.

Approach taken	Result, and why it stopped short
Handle the majority of instructions without invoking a model at all	**Succeeded and is in production.** Roughly seven in ten instructions are now answered without a model and respond effectively instantly. But this only removes the model from cases that were already unambiguous. The remaining three in ten are precisely those that require reasoning, and they are unchanged.
Train a model on our own usage to improve interpretation	**Improved substantially, then plateaued.** Further training stopped producing gains. The ceiling was the capacity of a model small enough to run here, not the amount of data available.
Test directly whether a larger model would help	**Confirmed that it would.** A comparable larger model scored measurably better on the same evaluation. It cannot be used in production because it does not fit alongside everything else the system needs.
Remove every avoidable cost from startup and execution	**Succeeded.** Startup time was reduced by roughly 80% and common operations are now effectively instantaneous. This exhausted the software-side savings. What remains is model inference time, which is hardware-bound.
Lower quality expectations to fit the hardware	**Rejected.** Rather than emit unusable output, the system detects it and reports honest failure. This preserves trust but does not deliver the capability.

The conclusion. Every avenue not requiring hardware has been taken. Each either succeeded and revealed the same underlying limit, or failed for the same reason. The performance and capability available to the product are bounded by how large a model the machine can hold, and no further amount of engineering changes that number.

5. Why cloud compute does not resolve this

Cloud services are the natural first suggestion, and for many machine learning projects they would be the correct answer. They are not the answer here, for reasons specific to what this product does. It is worth separating three distinct claims, because only two of them are absolute.

What cloud genuinely cannot do

VEGA's purpose is to operate *the user's own machine* — open their files, launch their applications, drive their browser, read their screen, listen to their microphone. A cloud instance is a computer in a distant data centre. It has no access to any of those things. Running the system in the cloud would have it open files on a machine nobody is sitting at. **This is not a limitation that more cloud resources would fix**; it is a mismatch between what the product does and what a remote server is.

Where hosted notebooks specifically fail

Shared notebook services such as Google Colab carry three further obstacles, any one of which is disqualifying for a deployed product:

Constraint	Effect
Sessions are temporary and disconnect when idle, with hard limits measured in hours	A service that thirty people depend on cannot be restarted every few hours.
The environment has no display, no sound device and no access to any user's files	The automation half of the product cannot run there at all.
The terms of service prohibit using the platform to host services or run long-running processes	Deploying on it would risk losing access without notice, most likely under load.

Where cloud is capable, but strategically wrong

A rented cloud server — on AWS or a comparable provider — *could* technically run the reasoning half of the system. It is important not to overstate the case: that part is possible. It is nonetheless the wrong choice here, for two reasons.

First, it destroys the property the product is being sold on. The moment instructions and documents leave the customer's network, the privacy guarantee is gone, and what remains competes directly with hosted providers who are larger, cheaper and further ahead. Second, rented compute is a permanent recurring cost that scales with every customer, whereas hardware is bought once and the customer's own server carries the ongoing load.

Where cloud resources genuinely help. They are valuable for *experiments* rather than deployment: training our own components, and measuring how large a model is actually required before any significant purchase is committed. The offer of Colab access is useful and will be used for exactly that. It cannot, however, host the product or serve its users.

6. Serving thirty or more users, with privacy intact

The requirement that thirty or more people use the system simultaneously is achievable without giving up the property that makes it worth buying. The design that achieves both is a split one.

Part	Where it runs	What it does
Local agent	On each user's own computer	Performs the actual work — files, applications, browser, screen, voice. Small and fast, and requires no model of its own.
Shared reasoning server	On the organisation's own server, inside their network	Interprets instructions that need reasoning. One capable machine answers all users.

This arrangement delivers three things at once. Automation continues to work, because a local agent is present on each machine to carry it out. Privacy is fully preserved, because the reasoning server belongs to the customer and sits inside their network — nothing crosses the organisation's boundary. And the economics improve sharply, because thirty people share one capable machine instead of requiring thirty.

The existing design supports this better than might be expected. Because roughly seven in ten instructions are already answered without a model, most work stays entirely on the user's own machine and never touches the server at all. Only the minority that genuinely require reasoning cross the network, which keeps the shared server's load far lower than a count of thirty users would suggest.

7. Resources required

One clarification first, because it determines what should be purchased. **Graphics memory, not system memory, is the binding constraint.** System memory determines whether a large model can be loaded at all; graphics memory determines whether it runs at a usable speed. A large model held in system memory alone runs several times slower than the small model in use today. Both matter, but graphics memory is what converts into responsiveness.

A second practical point: **a laptop's graphics processor cannot be upgraded.** It is soldered to the board. The requirement is therefore a separate desktop machine — which is convenient, because the same machine serves as the development system now and as the shared reasoning server described in section 6 later. One purchase, two purposes.

Proposed in three stages, smallest first

Stage	What it is	What it unblocks
1. Development and pilot machine	One desktop workstation with approximately 24 GB of graphics memory and 64 GB of system memory.	Models roughly ten times the current size at conversational speed. Expected to reduce the twelve-second delay to about one second, restore full screen understanding, allow specialised roles to run concurrently, and make reliable code generation possible. Unblocks development immediately and can serve a small pilot group.
2. Measurement before major spend	Cloud or Colab access, already offered, used for training our own components and benchmarking larger models.	Establishes on real workloads how large a model is genuinely required, so that stage 3 is sized from evidence rather than estimate. Costs a fraction of what it de-risks.
3. Production reasoning server	Server-class graphics memory sized by the findings from stage 2, with corresponding system memory.	Serves thirty or more concurrent users on frontier open models, inside the customer's own network.

The minimum that unblocks work immediately is stage 1. Stage 2 is deliberately placed before stage 3 so that the largest expenditure is made against measured evidence. If only one item can be approved, stage 1 is the one that converts directly into progress.

8. Expected outcome

Objective	Today	After stage 1
Response on instructions requiring reasoning	~12 seconds	~1 second
Screen understanding	Degraded fallback	Full capability
Specialised roles running together	One at a time	Several
Reliable code and long-form output	Not achievable	Achievable
Multi-step instructions	Under half succeed	Substantially improved

The response-time figure is the commercially important one. A one-second response is the difference between a tool someone reaches for and one they work around, and it is the change a customer notices first.

Summary. A working, verified product exists. The engineering routes to further progress that did not require hardware have all been taken and are exhausted. Cloud compute cannot substitute, because the product operates the user's own machine and because sending customer data outside their network would discard the property it is sold on. What remains is bounded by how large a model can be run, and that now limits speed, capability, and the multi-user requirement that has been set. One desktop workstation unblocks development immediately; the larger server is proposed only after measurement justifies its size.

VEGA — Autonomous System Controller — vision and resource proposal, v1.0.